Lab 10: Introduction to APIs: Wikipedia

INFO 4617/5617 · Week 10 · Chapter 10, Introduction to APIs: Wikipedia

This week every class is live coding: one lab in three parts, one a day. Each step says what we're building and why. Its code cells hold a comment that says what to write, and we write it together. Cells that start `# Given:` are already written: we read them together and run them. Nothing here is turned in.

We ask Wikipedia's APIs for one article's public record. It's the chapter's case, University of Colorado Boulder:

- **Part 1, Monday:** one request, read a level at a time; what a refused request looks like; then every edit since 2003, a page at a time;
- **Part 2, Wednesday:** who writes the article, who reads it, and whether the two move together;
- **Part 3, Friday:** what a pageview counts; the article's categories, languages and links; and a client library that does the paging for us.

Short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Friday's Step 16 uses mwclient, which `webdata` doesn't include. Install it once before Friday, in a terminal: `conda install -n webdata -c conda-forge mwclient`. (The chapter installs it from inside a notebook, with `%conda install -y -c conda-forge mwclient`.)

On Wednesday and Friday, start at that day's setup step.

> **Instructor key.** Rebuilt from scratch on 2026-11-10 for the next offering, from chapter 10 in the textbook at commit `3036573`, after the audit in issue #169. Run on 2026-11-10 from 07:52 to 08:30 UTC, in Python 3.14 with mwclient 0.11.0 (pip's; the course's conda command installs 0.10.1, whose source Step 16's cell was checked against, but which wasn't run). Wikimedia answered most requests from the course's cloud machine with 429, and in this lab a refusal often surfaces one cell after its request, where `labs/tools/run_key.py`'s retry of that cell alone can't help. So the key was run by a copy of its procedure that went back to the request cell and waited a minute each time, 30 minutes of waiting in all. Every cell ran and none is tagged `not-run-live`, but Step 16's cell keeps the output of the same code from an earlier run that morning (07:47 UTC), because the later run's 30 minutes were spent before it reached Step 16. The saved copies in `labs/week-10/data/` were written by this run's own cells, with `saved` and the `to_csv()` paths pointed at that local folder for the run and then set back to `main`. A rerun should match the edits (2,988 up to 2026-09-30, unless some are deleted), the pageviews (2,099 days and 1,367,292 views; the agents' months, unless Wikimedia reclassifies them) and Step 16's 91 edits in 2023. It won't necessarily match what describes the article today: its length (95,029 bytes) and its talk page's (682), its 40 categories and 38 languages, its 652 links, the 3,314 pages that link to it, and its 72 redirects.

Part 1 — Monday: ask one question, read the answer, turn the page

An **API** (application programming interface) is a door a site builds for programs: we ask in a documented form, and it answers in data, not in a page laid out for people. Today we send Wikipedia's Action API one request and read its answer a level at a time, see what a refused request looks like, and then fetch every edit to the article, 500 at a time.

Step 0 — Set up (4 minutes)

Put your email address in `CLIENT`, and run the given cell. It builds the User-Agent that Wikimedia's User-Agent policy asks for: a name and version, a way to reach you, then the library and its version. `HEADERS` sends it with every request. An **f-string**, a string with `f` before its quote, fills each `{name}` in it with that name's value. `API` is the Action API's address. See the chapter's "The Anatomy of an API Call".

```
# Given: put your email address in CLIENT, then run this cell
import requests
import time
import pandas as pd
import matplotlib.pyplot as plt

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
API = "https://en.wikipedia.org/w/api.php"
ARTICLE = "University of Colorado Boulder"
print(HEADERS)
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2'}
```

💬 Discuss

- In week 2 we wrote a User-Agent for a scraper. What does this one add, and who reads it?
- Wikimedia's rate limits count a client's requests a minute, by who it says it is and where it connects from, and everyone in this room shares one network. What happens if all of us run a loop at once?
- Wikipedia's `robots.txt` (week 2) disallows `/w/`, yet the API lives at `/w/api.php`. Why isn't that a contradiction?

Step 1 — One request, four parts (7 minutes)

Every request to the Action API has four parts: the base address, an `action` (what to do), parameters that narrow it, and a `format`. We write the parameters as a **dictionary**, pairs of a key and a value inside `{}`, and `requests` adds them to the address. Ask for `prop=info`, the article's basic facts. Before reading the answer, print its status code and its first 200 characters.

```
# info_params: "action": "query", "titles": ARTICLE, "prop": "info", "format": "json"
info_params = {"action": "query", "titles": ARTICLE, "prop": "info", "format": "json"}
```

```
# Send it as info_response, with params=info_params and headers=HEADERS
# Print its status_code and its first 200 characters, text[:200]
info_response = requests.get(API, params=info_params, headers=HEADERS)
print(info_response.status_code, info_response.text[:200])
```

```
200 {"batchcomplete":"","query":{"pages":{"289672":{"pageid":289672,"ns":0,"title":"University of
 ↪  Colorado Boulder","contentmodel":"wikitext","pagelanguage":"en","pagelanguagehtmlcode":"en",
 ↪  "pagelanguaged
```

```
# Print the address requests built from the four parts: info_response.url
print(info_response.url)
```

```
https://en.wikipedia.org/w/api.php?action=query&titles=University+of+Colorado+Boulder&prop=info&
 ↪  format=json
```

💬 Discuss

- Our first request in week 1 fetched this article as HTML. What came back this time, and what didn't we have to do that week 6's scraping did?

- Find the four parts in the address (week 5's URL anatomy). Which would you change to ask about another article?
- The status was 200. Which statuses would say the mistake was ours, and which the server's (week 5)?

Step 2 — Read the JSON one level at a time (10 minutes)

`.json()` turns the answer's text into Python dictionaries and lists. Go down one level a cell, printing the **keys** (the names on the left of each pair) before choosing one, as the chapter's "Common Issues to Debug" says. Under `pages`, the key is the page's ID, which we can't know in advance, so a given line takes the first (and only) value, with `list(pages.values())[0]`. The chapter shows the same nesting in a browser.

```
# info: the answer as Python, info_response.json(); print its keys()
info = info_response.json()
print(info.keys())
```

```
dict_keys(['batchcomplete', 'query'])
```

```
# Go down one level: print the keys of info["query"]
print(info["query"].keys())
```

```
dict_keys(['pages'])
```

```
# pages: info["query"]["pages"]; print its keys
pages = info["query"]["pages"]
print(pages.keys())
```

```
dict_keys(['289672'])
```

```
# Given: take the first (and only) page under pages, whatever its ID, and print it
info_page = list(pages.values())[0]
print(info_page)
```

```
{'pageid': 289672, 'ns': 0, 'title': 'University of Colorado Boulder', 'contentmodel':
 ↪  'wikitext', 'pagelanguage': 'en', 'pagelanguagehtmlcode': 'en', 'pagelanguagedir': 'ltr',
 ↪  'touched': '2026-11-08T21:17:43Z', 'lastrevid': 1382916123, 'length': 95029}
```

```
# Print the page's "title" and its "length" in bytes
print(info_page["title"], info_page["length"])
```

```
University of Colorado Boulder 95029
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: `titles` takes several titles at once, joined with `|`. Ask about three: the article, its talk page (`Talk:` before the title), and a misspelling of your choice. Type the titles; the given cell sends the request and prints each page's key, title and length. A **`for` loop** runs its indented lines once for each item, here each key under `pages`.

Share: the talk page's length, and the key the misspelling got.

```
# Microsprint: sprint_titles: ARTICLE + "|Talk:" + ARTICLE + "|" + a misspelling of the title
sprint_titles = ARTICLE + "|Talk:" + ARTICLE + "|University of Colorado Bolder"
```

```
# Given: ask prop=info about sprint_titles; print each page's key, title, and length (or
 ↪  "missing")
sprint_params = {"action": "query", "titles": sprint_titles, "prop": "info", "format": "json"}
sprint_response = requests.get(API, params=sprint_params, headers=HEADERS)
```

```
print(sprint_response.status_code)
sprint_pages = sprint_response.json()["query"]["pages"]
for key in sprint_pages:
    print(key, sprint_pages[key]["title"], sprint_pages[key].get("length", "missing"))
```

```
200
-1 University of Colorado Bolder missing
289672 University of Colorado Boulder 95029
2634293 Talk:University of Colorado Boulder 682
```

💬 Discuss

- Why is the page ID a key rather than a value? What key did the misspelling get, and what would `["length"]` have done with it?
- In week 6 we scraped Wikipedia's infoboxes with BeautifulSoup. Which approach breaks first when Wikipedia redesigns its pages?
- The talk page is shorter than the article, though editors have argued on it for years. Where did its old discussions go, and what would you need to know before comparing two pages by length?

Step 3 — When a request goes wrong (7 minutes)

A refused request still comes back, so the habit is to print its status and text before reading it. Send Step 1's request without `headers=`: `requests` then sends its own default User-Agent, which Wikimedia refuses with a 403. A 429 means too many requests: its `Retry-After` header says how many seconds to wait. A misspelled parameter comes back 200, with a warning inside (Errors and warnings).

```
# no_ua_response: Step 1's request without headers=; print its status and first 200 characters
no_ua_response = requests.get(API, params=info_params)
print(no_ua_response.status_code, no_ua_response.text[:200])
```

```
403 Please set a user-agent and respect our robot policy https://w.wiki/4wJS. See also
 ↪  https://phabricator.wikimedia.org/T400119.
```

```
# Given: when a cell stops with JSONDecodeError or KeyError, print its response so (here, Step
 ↪  1's)
print(info_response.status_code, info_response.headers.get("retry-after"))
print(info_response.text[:300])
```

```
200 None
{"batchcomplete":"","query":{"pages":{"289672":{"pageid":289672,"ns":0,"title":"University of
 ↪  Colorado
 ↪  Boulder","contentmodel":"wikitext","pagelanguage":"en","pagelanguagehtmlcode":"en","pagelang
 ↪  uagedir":"ltr","touched":"2026-11-08T21:17:43Z","lastrevid":1382916123,"length":95029}}}}
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: break Step 1's request on purpose, one way: misspell a key (`"title"` for `"titles"`), leave out `"format"`, or ask for a `prop` that doesn't exist. Send it with `HEADERS`, and print its status and first 200 characters.

Share: what you broke, the status, and what the answer said. Which line of Step 2 would have stopped, and with what error?

```
# Microsprint: broken_params: Step 1's dictionary with one thing broken; send it as
 ↪  broken_response
# Print its status and first 200 characters
```

```
broken_params = {"action": "query", "title": ARTICLE, "prop": "info", "format": "json"}
broken_response = requests.get(API, params=broken_params, headers=HEADERS)
print(broken_response.status_code, broken_response.text[:200])
```

```
200 {"warnings":{"main":{"*":"Unrecognized parameter: title."}},"batchcomplete":""}
```

💬 **Discuss**

- Which of the failures we saw came back with status 200? Why would an API answer ”OK” to a request it couldn't do?
- Of 403 and 429, which is about who you say you are, and which about how often you ask?
- The chapter's functions call `raise_for_status()`, which stops on a status of 400 or more. Which of today's failures would it have caught, and which not?

Step 4 — Two pages by hand (13 minutes)

`prop=revisions` lists an article's edits, newest first, at most 500 a request. `rvstart` fixes the newest edit we accept, so a rerun gets the same answer. When there are more, the answer carries a `"continue"` key: add its pairs to the parameters and ask again. `.copy()` makes a second dictionary, so page one's stays as it was, and `.update()` adds another dictionary's pairs to it. See ”Getting Revisions”.

```
# Given: the parameters for the article's edits up to the end of September 2026, newest first
rev_params = {"action": "query", "titles": ARTICLE, "prop": "revisions",
              "rvprop": "ids|timestamp|user|size|comment",
              "rvstart": "2026-09-30T23:59:59Z", "format": "json"}
```

```
# Ask for the most a request allows: set rev_params["rvlimit"] to "max"
rev_params["rvlimit"] = "max"
```

```
# Page one: send rev_params as first_response, and read its JSON as first_page
# Print first_page["continue"]
first_response = requests.get(API, params=rev_params, headers=HEADERS)
first_page = first_response.json()
print(first_page["continue"])
```

```
{'rvcontinue': '20191027230600|923341341', 'continue': '||'}
```

```
# second_params: rev_params.copy(), then .update() it with first_page["continue"]; print it
second_params = rev_params.copy()
second_params.update(first_page["continue"])
print(second_params)
```

```
{'action': 'query', 'titles': 'University of Colorado Boulder', 'prop': 'revisions', 'rvprop':
 ↪  'ids|timestamp|user|size|comment', 'rvstart': '2026-09-30T23:59:59Z', 'format': 'json',
 ↪  'rvlimit': 'max', 'rvcontinue': '20191027230600|923341341', 'continue': '||'}
```

```
# Page two: send second_params as second_response, and read its JSON as second_page
# Print second_page["continue"]
second_response = requests.get(API, params=second_params, headers=HEADERS)
second_page = second_response.json()
print(second_page["continue"])
```

```
{'rvcontinue': '20140710145313|616384704', 'continue': '||'}
```

```
# Given: how many edits each page holds, and the oldest edit on it
first_edits = list(first_page["query"]["pages"].values())[0]["revisions"]
print("Page one:", len(first_edits), "edits, back to", first_edits[-1]["timestamp"])
second_edits = list(second_page["query"]["pages"].values())[0]["revisions"]
print("Page two:", len(second_edits), "edits, back to", second_edits[-1]["timestamp"])
```

```
Page one: 500 edits, back to 2019-10-29T16:59:07Z
Page two: 500 edits, back to 2014-07-10T19:44:47Z
```

💬 Discuss

- Page one held exactly 500 edits. Left out, `rvlimit` gives 10 a page, still with a `"continue"` and no warning. Why is a round number a warning sign?
- Sent with `rev_params` instead of `second_params`, page two's request gets page one again, with status 200 and no warning. How would you notice?
- Why fix `rvstart` at a date in the past? What does that give someone who reruns your notebook next year?

Step 5 — Every edit: the loop, given (9 minutes)

`get_revisions()` is the chapter's function, given. A **function** is named lines that run when called. Find your two moves in it: the request, and the update from `"continue"`; `while True` repeats them until `break`. It pauses a second between requests, and leaves out the chapter's `raise_for_status()`. Then a given cell numbers the editors instead of naming them, drops names, addresses and summaries, and saves a copy. See "Understanding Pagination".

```
# Given: the chapter's get_revisions(), with a one-second pause and without raise_for_status()
def get_revisions(title, until="2026-09-30T23:59:59Z"):
    params = {"action": "query", "titles": title, "prop": "revisions",
              "rvprop": "ids|timestamp|user|size|comment",
              "rvlimit": "max", "rvstart": until, "format": "json"}
    all_revisions = []
    while True:
        response = requests.get(API, params=params, headers=HEADERS)
        data = response.json()
        pages = data["query"]["pages"]
        page_id = list(pages.keys())[0]
        all_revisions.extend(pages[page_id].get("revisions", []))
        if "continue" in data:
            params.update(data["continue"])   # ask for the next batch
            time.sleep(1)
        else:
            break
    return pd.DataFrame(all_revisions)
```

```
# revisions: get_revisions(ARTICLE); print how many rows, and its oldest and newest "timestamp"
revisions = get_revisions(ARTICLE)
print(len(revisions), revisions["timestamp"].min(), revisions["timestamp"].max())
```

```
2988 2003-08-05T16:42:10Z 2026-09-09T12:55:42Z
```

```
# Given: number the editors by first edit (1 is the first) instead of naming them; make "anon"
 ↪  and "temp"
# True or False; keep only the columns below (no names, IP addresses, summaries or IDs); save as
 ↪  revisions.csv
no_names = revisions.sort_values("timestamp", ignore_index=True)
no_names["editor"] = pd.factorize(no_names["user"])[0] + 1
no_names["anon"] = no_names["anon"].notna()
no_names["temp"] = no_names["temp"].notna()
no_names = no_names[["timestamp", "size", "anon", "temp", "editor"]]
no_names.to_csv("revisions.csv", index=False)
print(no_names.tail(3))
```

```
                timestamp   size   anon   temp  editor
2985  2026-09-04T04:47:10Z  94887  False   True    1414
2986  2026-09-04T21:30:31Z  95179  False  False    1357
2987  2026-09-09T12:55:42Z  95255  False  False    1415
```

```
# Given, only if a cell above stopped: remove the "#" from the next two lines (saved 2026-11-10)
# saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-10/data/"
# no_names = pd.read_csv(saved + "revisions.csv")
```

💬 **Discuss**

- Our loop assumes every request works. If its fourth request got a 429, which line would stop, and what would we have lost?
- Every username is public on the article's View history. Why number the editors in a copy we publish, rather than name them?
- A saved copy is one day's answer. What can it not tell you that the live request can?

Part 2 — Wednesday: who writes the article, and who reads it

On Monday we fetched every edit. Today we count them by month and by editor, ask a second API how many people read the article each day, and check whether editing and reading move together.

Step 6 — Set up again (4 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Put your email address in the given setup cell and run it; it also sets `PAGEVIEWS`, the pageviews API's address, and `saved`, the address of the course's saved copies. The next cell loads Monday's table: every edit to the article up to September 30, 2026, with each editor as a number, not a name. `anon` and `temp` say whether an edit came from an IP address or a temporary account.

```
# Given: Part 2's setup. Put your email address in CLIENT, then run it
import requests
import time
import pandas as pd
import matplotlib.pyplot as plt

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
API = "https://en.wikipedia.org/w/api.php"
PAGEVIEWS = "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
ARTICLE = "University of Colorado Boulder"
saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-10/data/"
```

```
# Given: load Monday's table, saved by the course on 2026-11-10 with Step 5's code
revisions = pd.read_csv(saved + "revisions.csv")
```

```
# Print how many rows revisions has, and its first three rows, head(3)
print(len(revisions))
print(revisions.head(3))
```

```
2988
             timestamp  size   anon   temp  editor
0  2003-08-05T16:42:10Z  1037  False  False       1
1  2003-08-20T20:03:52Z  1171  False  False       1
2  2003-12-23T07:08:15Z  1232  False  False       2
```

💬 **Discuss**

- The table has numbers where the article's history has names. What can you still count, and what can't you do any more?
- Nobody sent a request to load it. Whose server did the table come from, and what would you check before trusting a saved copy you found online?

Step 7 — Edits per month (9 minutes)

The timestamps are text. `pd.to_datetime()` turns them into dates, here in UTC. With the dates as the table's **index**, the labels down its left side, `resample("ME")` groups the edits by month and labels each month by its last day; `.size()` counts each group. The given cell plots it. See "Analyzing Revision History".

```
# Turn revisions["timestamp"] into dates with pd.to_datetime()
revisions["timestamp"] = pd.to_datetime(revisions["timestamp"])
```

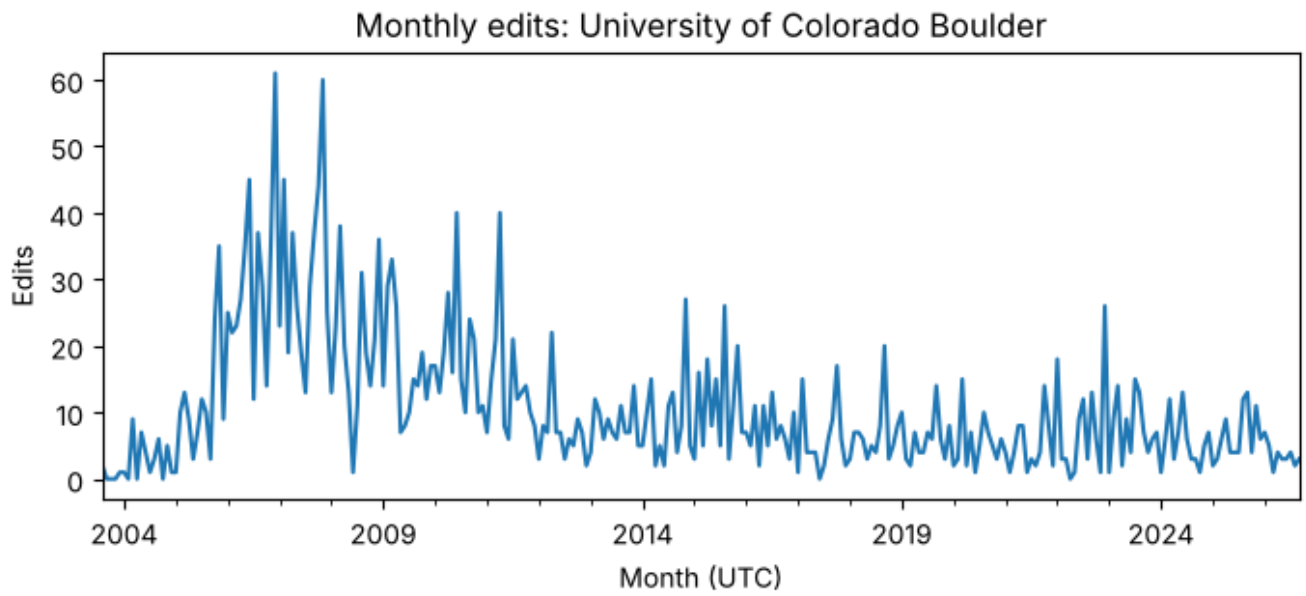
```
# by_time: revisions.set_index("timestamp")
# monthly_edits: by_time.resample("ME").size(), the edits per month; print the last three
by_time = revisions.set_index("timestamp")
monthly_edits = by_time.resample("ME").size()
print(monthly_edits.tail(3))
```

```
timestamp
2026-07-31 00:00:00+00:00    4
2026-08-31 00:00:00+00:00    2
2026-09-30 00:00:00+00:00    3
Freq: ME, dtype: int64
```

```
# Print the busiest month, monthly_edits.idxmax(), and its number of edits, monthly_edits.max()
print(monthly_edits.idxmax(), monthly_edits.max())
```

```
2006-12-31 00:00:00+00:00 61
```

```
# Given: plot the monthly edits, with a title and labeled axes
fig, ax = plt.subplots(figsize=(8, 3))
monthly_edits.plot(ax=ax)
ax.set_title(f"Monthly edits: {ARTICLE}")
ax.set_xlabel("Month (UTC)")
ax.set_ylabel("Edits")
plt.show()
```



⏱️ Microsprint (3 minutes, alone or in pairs)

Do: count by year instead of by month: `"YE"` in place of `"ME"`. Which year had the most edits?

Share: the busiest year and its number of edits.

```
# Microsprint: yearly_edits: by_time.resample("YE").size(); print its idxmax() and max()
yearly_edits = by_time.resample("YE").size()
print(yearly_edits.idxmax(), yearly_edits.max())
```

```
2007-12-31 00:00:00+00:00 377
```

💬 Discuss

- In the key's run, the busiest month was December 2006, with 61 edits, and the first five years held about a third of all the edits. What does that suggest about how an article matures?
- A busy month might be one long argument or many small fixes. What would you read to tell them apart, and which column did the saved copy drop?

Step 8 — Who edits (9 minutes)

`nunique()` counts the different values in a column: here, editors. `value_counts()` counts each one's edits, most first, and `head(10)` keeps the top ten. Their total divided by all the edits is the top ten's share: is the work in a few hands, or many? The chapter's "Analyzing Revision History" prints the same ten as names; ours are numbers.

```
# Print how many different editors there are: revisions["editor"].nunique()
print(revisions["editor"].nunique())
```

```
1415
```

```
# edit_counts: revisions["editor"].value_counts(); print its first ten, head(10)
edit_counts = revisions["editor"].value_counts()
print(edit_counts.head(10))
```

```
editor
64      92
154     82
```

```
485     78
1002    49
888     40
394     37
3       27
102     24
964     23
4       22
Name: count, dtype: int64
```

```
# top_ten: edit_counts.head(10); print their share of all edits, top_ten.sum() / len(revisions)
top_ten = edit_counts.head(10)
print(top_ten.sum() / len(revisions))
```

```
0.15863453815261044
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: count the edits made without an account, from an IP address (`anon`) or a temporary account (`temp`). The given cell keeps those rows as `no_account_edits`. Print what share of all edits they are, and how many different editor numbers they have. (On another article, `anon` and `temp` are there only if at least one edit has them.) A short version of Additional Exercise 6's question of whether anonymous editors count.

Share: the share, and the number of editors without an account.

```
# Given: the rows whose edit came without an account ("|" means "or")
no_account = revisions["anon"] | revisions["temp"]
no_account_edits = revisions[no_account]
```

```
# Microsprint: print len(no_account_edits) / len(revisions), the share of all edits;
# then no_account_edits["editor"].nunique(), the number of editors
print(len(no_account_edits) / len(revisions))
print(no_account_edits["editor"].nunique())
```

```
0.35240963855421686
676
```

💬 Discuss

- Is this article written by a dedicated few or a broad crowd? Which number above tells you?
- Each IP address counts as its own editor, and one person can use many. What does that do to the count of editors?
- Usernames and IP addresses are public on the article's history. By week 2's questions about privacy, is it fine to publish a ranking of an article's editors by name?

Step 9 — A second API for readers (10 minutes)

Pageviews come from Wikimedia's pageviews API, whose parameters are parts of the address's path, not a dictionary: project, access, **agent** (who is reading: `user` is people), article, granularity, start and end. Titles in the path should have underscores for spaces. Build the address with an f-string, print what came back, and make a table of its `"items"`. Its timestamps are `YYYYMMDDHH`, so we give that `format`. See "Getting Pageviews".

```
# article: ARTICLE.replace(" ", "_"); print it
article = ARTICLE.replace(" ", "_")
print(article)
```

```
University_of_Colorado_Boulder
```

```
# views_path: f"{PAGEVIEWS}/en.wikipedia/all-access/user/{article}"
# views_url: views_path + "/daily/20210101/20260930"; print it
views_path = f"{PAGEVIEWS}/en.wikipedia/all-access/user/{article}"
views_url = views_path + "/daily/20210101/20260930"
print(views_url)
```

```
https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/en.wikipedia/all-access/user/Uni
↪ versity_of_Colorado_Boulder/daily/20210101/20260930
```

```
# views_response: requests.get(views_url, headers=HEADERS); print its status and text[:200]
views_response = requests.get(views_url, headers=HEADERS)
print(views_response.status_code, views_response.text[:200])
```

```
200 {"items":[{"project":"en.wikipedia","article":"University_of_Colorado_Boulder","granularity"
↪ :"daily","timestamp":"2021010100","access":"all-access","agent":"user","views":559},{"projec
↪ t":"en.wikipedia
```

```
# views_data: views_response.json(); print its keys
views_data = views_response.json()
print(views_data.keys())
```

```
dict_keys(['items'])
```

```
# views: pd.DataFrame(views_data["items"]); print how many days, and the sum of views["views"]
views = pd.DataFrame(views_data["items"])
print(len(views), views["views"].sum())
```

```
2099 1367292
```

```
# Turn views["timestamp"] into dates: pd.to_datetime(views["timestamp"], format="%Y%m%d%H")
views["timestamp"] = pd.to_datetime(views["timestamp"], format="%Y%m%d%H")
```

```
# Given: save views in this folder as daily-views.csv, as the course did for its saved copy
views.to_csv("daily-views.csv", index=False)
```

```
# Given, only if a cell above stopped: remove the "#" from the next line (saved 2026-11-10)
# views = pd.read_csv(saved + "daily-views.csv", parse_dates=["timestamp"])
```

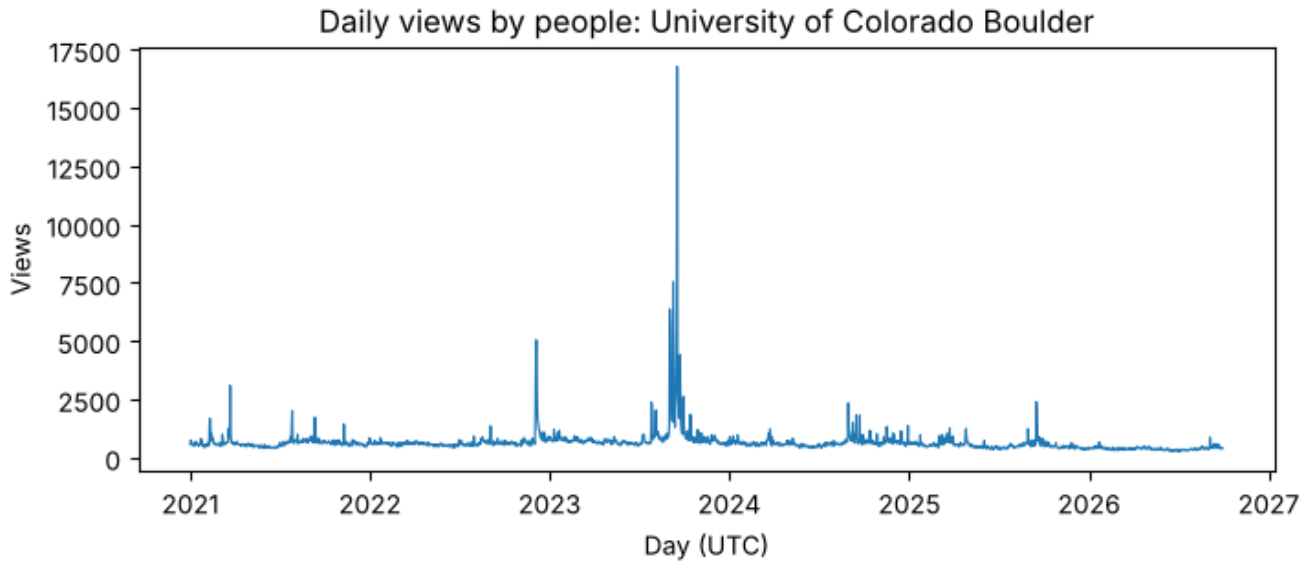
💬 Discuss

- Week 5 called this a path-segment API, and the Action API a query-parameter one. Why would a title with a slash, such as AC/DC, break this address but not Step 1's request?
- Why didn't this request need a `"continue"`?
- In week 1 we asked this API for `all-agents`. What changes when we ask for `user`?

Step 10 — The biggest day (7 minutes)

The given cell plots the daily views. `idxmax()` gives the row label of the largest value, and `.loc[]` reads the row with that label. The API's days are UTC days, so an evening in Colorado counts toward the next day. The chapter's "Getting Pageviews" says what made the biggest spikes.

```
# Given: plot the daily views, with a title and labeled axes
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(views["timestamp"], views["views"], linewidth=0.8)
ax.set_title(f"Daily views by people: {ARTICLE}")
ax.set_xlabel("Day (UTC)")
ax.set_ylabel("Views")
plt.show()
```




```
# peak: views["views"].idxmax(), the row with the most views; print views.loc[peak]
peak = views["views"].idxmax()
print(views.loc[peak])
```

```
project                          en.wikipedia
article        University_of_Colorado_Boulder
granularity                             daily
timestamp                2023-09-17 00:00:00
access                           all-access
agent                                  user
views                                 16767
Name: 989, dtype: object
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: compare the peak with an ordinary day: print the median of `views["views"]`, the number of views that half the days were above.

Share: the median, and about how many ordinary days the peak was worth.

```
# Microsprint: print the median of views["views"]
print(views["views"].median())
```

```
590.0
```

💬 Discuss

- The peak was 16,767 views on 2023-09-17, a UTC day. The chapter says the game behind it ended late on the night of the 16th in Colorado. What mistake could you make reporting the peak as a Colorado date?
- How would you check what happened that day with data, such as the article's edits around it, rather than from memory?
- A spike measures attention, not approval. Which claims can it support, and which can't it?

Step 11 — Two monthly series that don't line up (11 minutes)

Do edits and readers move together? Count views per month with `resample("ME").sum()`, then compare the last labels of both series. Both are month ends, but the edits' carry a time zone (`+00:00`) and the views' don't, so pandas refuses to line them up, with `TypeError: Cannot join tz-naive with tz-aware`

`DatetimeIndex`. `tz_localize(None)` drops it. Then `corr()` says how closely they move: 1 together, 0 not at all. See "Combining Endpoints".

```
# views_by_day: views.set_index("timestamp")
# monthly_views: views_by_day["views"].resample("ME").sum(); print the last two
views_by_day = views.set_index("timestamp")
monthly_views = views_by_day["views"].resample("ME").sum()
print(monthly_views.tail(2))
```

```
timestamp
2026-08-31    11890
2026-09-30    14000
Freq: ME, Name: views, dtype: int64
```

```
# Print the last two of monthly_edits, and compare their labels with monthly_views'
print(monthly_edits.tail(2))
```

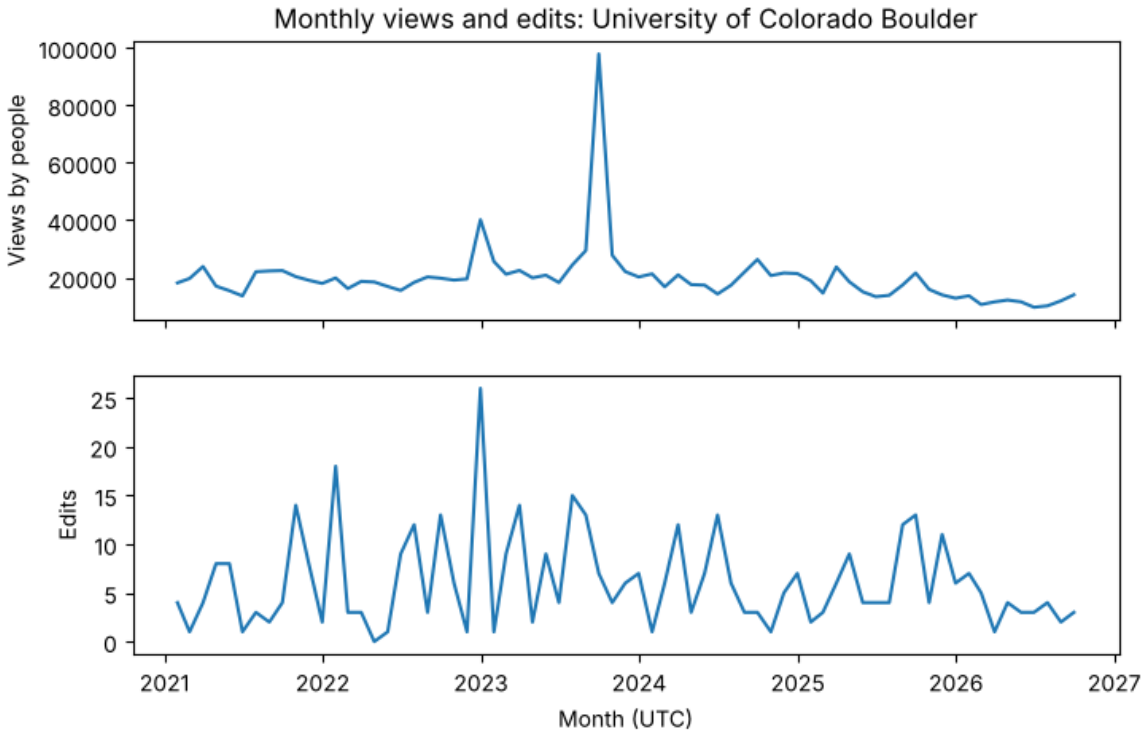
```
timestamp
2026-08-31 00:00:00+00:00    2
2026-09-30 00:00:00+00:00    3
Freq: ME, dtype: int64
```

```
# naive_edits: monthly_edits.tz_localize(None); edits_since_2021: naive_edits.loc["2021":]
naive_edits = monthly_edits.tz_localize(None)
edits_since_2021 = naive_edits.loc["2021":]
```

```
# Print how closely they move together: monthly_views.corr(edits_since_2021)
print(monthly_views.corr(edits_since_2021))
```

```
0.20929700242658758
```

```
# Given: monthly views above, monthly edits below, on one time axis
fig, axes = plt.subplots(2, 1, figsize=(8, 5), sharex=True)
axes[0].plot(monthly_views.index, monthly_views.values)
axes[0].set_title(f"Monthly views and edits: {ARTICLE}")
axes[0].set_ylabel("Views by people")
axes[1].plot(edits_since_2021.index, edits_since_2021.values)
axes[1].set_ylabel("Edits")
axes[1].set_xlabel("Month (UTC)")
plt.show()
```



💬 **Discuss**

- The edits and the views came from two APIs. Why do their month labels differ, and why is it better that pandas refused than guessed?
- In the key's run the correlation was 0.21. What does that say about editing and reading, and what can't a correlation say?
- Find December 2022 and September 2023 in the plot; the chapter's ”Combining Endpoints” says what happened. Why would one month bring editors and the other mostly readers?

Part 3 — Friday: what the counts count, and what else is there

A pageview is not a person, so first we split the views by who, or what, was reading. Then the rest of the article's record: its categories, languages and links, with the chapter's two functions for long lists. Last, a client library that does the paging for us.

Step 12 — Set up again (3 minutes)

Part 3 starts fresh. Put your email address in the given setup cell and run it, then the cell that loads Monday's table, which Step 16 checks a client library against.

```
# Given: Part 3's setup. Put your email address in CLIENT, then run it
import requests
import time
import pandas as pd
import matplotlib.pyplot as plt

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
API = "https://en.wikipedia.org/w/api.php"
PAGEVIEWS = "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
ARTICLE = "University of Colorado Boulder"
saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-10/data/"
```

```
# Given: load Monday's table, saved by the course on 2026-11-10, with its timestamps as dates
revisions = pd.read_csv(saved + "revisions.csv", parse_dates=["timestamp"])
print(len(revisions), "edits")
```

```
2988 edits
```

💬 **Discuss**

- On Monday the loop sent six requests to get this table; today, none. When would you rather have the saved copy, and when the live request?
- Which of today's numbers will come live, and which from Monday's saved table?

Step 13 — Who is reading? (10 minutes)

Wikimedia's definition of a pageview splits the counts by **agent**: `user` (people), `spider` (crawlers that say what they are) and `automated` (bots that don't). The given `get_pageviews()` is the chapter's: Step 9's lines as a function. In its first line, `agent="user"` is a **default**, used when a call doesn't give one. Call it for spiders, by month; then a given loop asks for all three. In 2025 Wikimedia moved past traffic from `user` to `automated`. See "What a Pageview Counts".

```
# Given: the chapter's get_pageviews(), without raise_for_status(): Step 9's lines as a function
def get_pageviews(title, start, end, agent="user", granularity="daily"):
    article = title.replace(" ", "_")
    url = ("https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
           f"/en.wikipedia/all-access/{agent}/{article}/{granularity}/{start}/{end}")
    response = requests.get(url, headers=HEADERS)
    views = pd.DataFrame(response.json()["items"])
    views["timestamp"] = pd.to_datetime(views["timestamp"], format="%Y%m%d%H")
    return views[["timestamp", "views"]]
```

```
# spider_views: get_pageviews(ARTICLE, "20240101", "20260930", agent="spider",
#   granularity="monthly"); print its last three rows, tail(3)
spider_views = get_pageviews(ARTICLE, "20240101", "20260930",
                             agent="spider", granularity="monthly")
print(spider_views.tail(3))
```

```
    timestamp  views
30 2026-07-01   2099
31 2026-08-01   9362
32 2026-09-01   2924
```

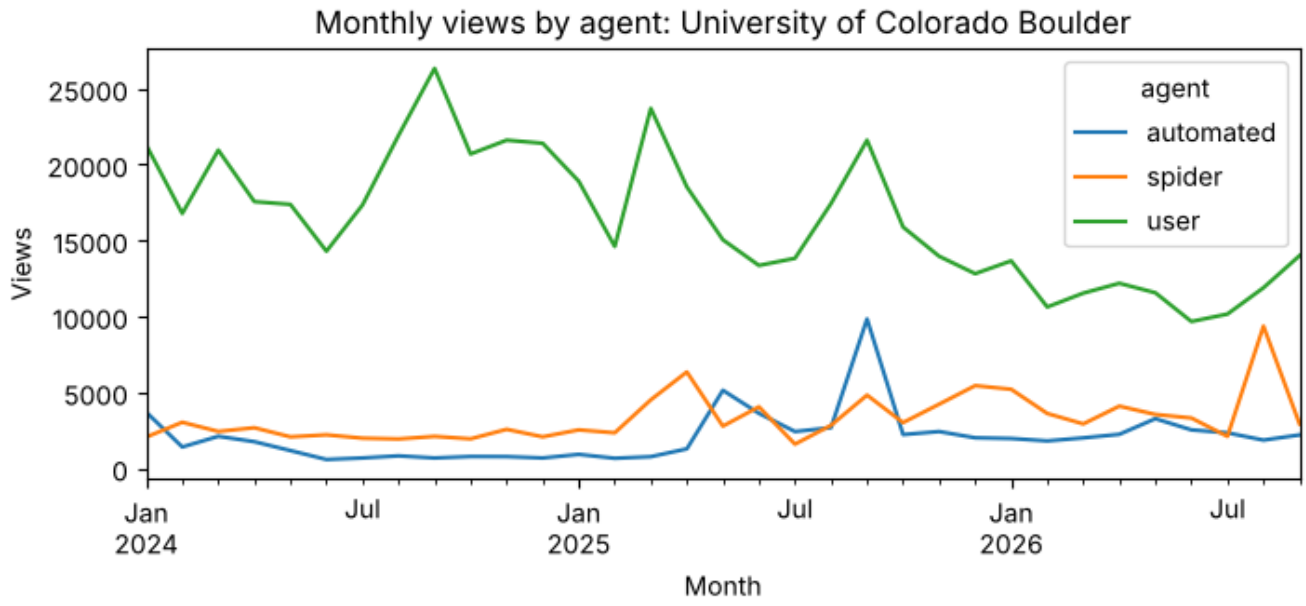
```
# Given: the same for each agent, a second apart; one column per agent; save it as by-agent.csv
agents = []
for agent in ["user", "spider", "automated"]:
    monthly = get_pageviews(ARTICLE, "20240101", "20260930",
                            agent=agent, granularity="monthly")
    monthly["agent"] = agent
    agents.append(monthly)
    time.sleep(1)
by_agent = pd.concat(agents).pivot(index="timestamp", columns="agent", values="views")
by_agent.to_csv("by-agent.csv")
print(by_agent.tail(3))
```

```
agent       automated  spider   user
timestamp
2026-07-01       2334    2099  10155
```

```
2026-08-01       1855    9362  11890
2026-09-01       2196    2924  14000
```

```
# Given, only if the cell above stopped: remove the "#" from the next line (saved 2026-11-10)
# by_agent = pd.read_csv(saved + "by-agent.csv", index_col="timestamp",
 ↪  parse_dates=["timestamp"])
```

```
# Given: plot the three agents, with a title and labeled axes
fig, ax = plt.subplots(figsize=(8, 3))
by_agent.plot(ax=ax)
ax.set_title(f"Monthly views by agent: {ARTICLE}")
ax.set_xlabel("Month")
ax.set_ylabel("Views")
plt.show()
```



⏱️ Microsprint (4 minutes, alone or in pairs)

Do: check the three lines. Ask `get_pageviews()` for `agent="all-agents"`, by month, for the same months. The given cell checks that each month is the sum of the three agents, and finds the month when people were the smallest share of the views.

Share: whether the sum checks out, and that month and its share.

```
# Microsprint: all_views: get_pageviews() for the same months, with agent="all-agents", monthly
all_views = get_pageviews(ARTICLE, "20240101", "20260930",
                          agent="all-agents", granularity="monthly")
```

```
# Given: is each month the sum of the three agents? And when were people the smallest share?
all_by_month = all_views.set_index("timestamp")["views"]
print("Sum checks out:", (by_agent.sum(axis=1) == all_by_month).all())
people_share = by_agent["user"] / all_by_month
print(people_share.idxmin(), round(people_share.min(), 2))
```

```
Sum checks out: True
2026-08-01 00:00:00 0.51
```

💬 Discuss

- In the key's run, people were 51% of the views in August 2026, their smallest share. How would a study that used `all-agents` have read that month?
- In 2025 Wikimedia moved past counts from one agent to another. What does that mean for anyone who downloaded the counts before then?
- Our Selenium visits in week 8 came from a real browser. Which agent should they count as?

Step 14 — Everything else is the same query (8 minutes)

Categories, language editions, links and what links here are all `action=query` with another `prop` or `list`, and they all page the same way. One request can ask for two `prop`s, joined with `|`, each with its own limit: `cllimit` and `lllimit`. A short list fits in one answer, but check for `"continue"` before trusting a count. Templates, external links and user contributions work the same way: see "From the Interface to the API".

```
# cat_lang_params: "action": "query", "titles": ARTICLE, "format": "json",
#   "prop": "categories|langlinks", "cllimit": "max", "lllimit": "max"
cat_lang_params = {"action": "query", "titles": ARTICLE, "format": "json",
                   "prop": "categories|langlinks", "cllimit": "max", "lllimit": "max"}
```

```
# Send it as cat_lang_response; cat_lang: its JSON; print whether "continue" is in cat_lang
cat_lang_response = requests.get(API, params=cat_lang_params, headers=HEADERS)
cat_lang = cat_lang_response.json()
print("continue" in cat_lang)
```

```
False
```

```
# Given: count the categories and language editions, and print the first three categories
cat_lang_page = list(cat_lang["query"]["pages"].values())[0]
print(len(cat_lang_page["categories"]), "categories,", len(cat_lang_page["langlinks"]),
 ↪  "languages")
for category in cat_lang_page["categories"][:3]:
    print(category["title"])
```

```
40 categories, 38 languages
Category:1876 establishments in Colorado Territory
Category:All Wikipedia articles written in American English
Category:All articles containing potentially dated statements
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: which other titles send readers here? `prop=redirects` isn't in the chapter: read its page for the name of its limit parameter, and set it to `"max"`. The given cell sends the request and prints how many, whether there's a `"continue"`, and the first eight. A short version of Additional Exercise 4.

Share: how many there are, and the most surprising title.

```
# Microsprint: redirect_params: "action": "query", "titles": ARTICLE, "format": "json",
#   "prop": "redirects", and the limit parameter from API:Redirects set to "max"
redirect_params = {"action": "query", "titles": ARTICLE, "format": "json",
                   "prop": "redirects", "rdlimit": "max"}
```

```
# Given: send redirect_params; print how many redirect here, whether there's more, and eight
redirect_response = requests.get(API, params=redirect_params, headers=HEADERS)
print(redirect_response.status_code)
redirect_data = redirect_response.json()
redirect_page = list(redirect_data["query"]["pages"].values())[0]
print(len(redirect_page["redirects"]), "redirects; more to come:", "continue" in redirect_data)
```

```
for redirect in redirect_page["redirects"][:8]:
    print(redirect["title"])
```

```
200
72 redirects; more to come: False
University of Colorado, at Boulder
University of Colorado, Boulder
University Memorial Center at the University of Colorado at Boulder
Tocqueville Initiative
University of colorado at boulder
University of Colorado in Boulder
University of Colorado - Boulder
Western Academic Talent Search
```

💬 **Discuss**

- The API counted 40 categories, more than the box at the foot of the article shows, and the first three above say why. What are hidden categories, and should a study of how Wikipedia classifies a topic include them? (`clshow` on API:Categories leaves them out.)
- Had you misspelled `cllimit`, the API would have sent 10 categories and a `"continue"`, with status 200. How would you have noticed?
- Thirty-eight other editions have an article on this topic. What could you learn by comparing them, as week 6 did with French infoboxes?

Step 15 — The chapter's two functions (7 minutes)

For long lists the chapter has two functions, given here. `query_all()` sends any query and follows `"continue"` to the end, as `get_revisions()` did; we added the pause. In it, `**params` copies one dictionary's pairs into another. `page_list()` uses it to collect one `prop` of one page. Call `page_list()` once, for the article's wikilinks (`pllimit`); a given cell then calls `query_all()` once, for what links here: every page in namespace 0, redirects included.

```
# Given: the chapter's query_all(), with a one-second pause and without raise_for_status()
def query_all(params):
    params = {"action": "query", "format": "json", **params}
    results = []
    while True:
        response = requests.get(API, params=params, headers=HEADERS)
        data = response.json()
        results.append(data["query"])
        if "continue" in data:
            params.update(data["continue"])
            time.sleep(1)
        else:
            break
    return results
```

```
# Given: the chapter's page_list(): one prop (links, categories, langlinks, ...) of one page, as
 ↪  a list
def page_list(title, prop, limit_param):
    items = []
    for batch in query_all({"titles": title, "prop": prop, limit_param: "max"}):
        for page in batch["pages"].values():
            items.extend(page.get(prop, []))
    return items
```

```
# links: page_list(ARTICLE, "links", "pllimit"); print how many, and the first three, links[:3]
links = page_list(ARTICLE, "links", "pllimit")
print(len(links), links[:3])
```

```
652 [{'ns': 0, 'title': '1000-Word Philosophy'}, {'ns': 0, 'title': '1996 NCAA conference
 ↪  realignment'}, {'ns': 0, 'title': '1stBank Center'}]
```

```
# Given: what links here, with query_all(); then save links.csv and backlinks.csv in this folder
backlinks = []
for batch in query_all({"list": "backlinks", "bltitle": ARTICLE, "bllimit": "max",
                        "blnamespace": 0}):
    backlinks.extend(batch["backlinks"])
print(len(backlinks), "pages link here;", sum("redirect" in b for b in backlinks), "are
 ↪  redirects")
pd.DataFrame(links).to_csv("links.csv", index=False)
pd.DataFrame(backlinks).to_csv("backlinks.csv", index=False)
```

```
3314 pages link here; 70 are redirects
```

```
# Given, only if a cell above stopped: remove the "#" from the next two lines (saved 2026-11-10)
# links = pd.read_csv(saved + "links.csv").to_dict("records")
# backlinks = pd.read_csv(saved + "backlinks.csv").to_dict("records")
```

💬 Discuss

- Typed as `"pllimt"`, the limit's name is ignored with a warning nobody prints: `page_list()` still gets every link, 10 a request, so about 66 requests instead of 2. How would you notice, and who pays for the difference?
- Some of the pages that link here are redirects. Would you call the rest ”articles that link here”? What does `blfilterredir` on API:Backlinks let you choose?
- In week 6 we parsed pages' HTML. Which of today's lists could you have scraped from the article itself, and which only the API gives?

Step 16 — The same with a client library (4 minutes)

mwclient, installed before class, follows `"continue"` for you, and adds its own name and version to `CLIENT` in the User-Agent. It's given: read it, and run it. Versions 0.10.1 (conda-forge's) and 0.11 (pip's) name the per-request size differently, so we pass neither, take the default of 50, and ask only for 2023's edits. mwclient speaks only the Action API, and stops on a 429. See ”From Raw Requests to a Client”.

```
# Given: the article's facts and its 2023 edits with mwclient; then the count from Monday's table
import mwclient
print("mwclient", mwclient.__version__)
site = mwclient.Site("en.wikipedia.org", clients_useragent=CLIENT)
print(site.connection.headers["User-Agent"])
mw_page = site.pages[ARTICLE]
print(mw_page.name, mw_page.pageid, mw_page.length)
edits_2023 = list(mw_page.revisions(start="2023-12-31T23:59:59Z",
                                    end="2023-01-01T00:00:00Z"))
print(len(edits_2023), "edits in 2023, from mwclient")
print((revisions["timestamp"].dt.year == 2023).sum(), "edits in 2023, from Monday's table")
```

```
mwclient 0.11.0
```

```
WebDataScience/1.0 (your-email@colorado.edu) mwclient/0.11.0
 ↪  (https://github.com/mwclient/mwclient)
University of Colorado Boulder 289672 95029
```

```
91 edits in 2023, from mwclient
91 edits in 2023, from Monday's table
```

💬 **Discuss**

- What did mwclient do for you that `get_revisions()` did in plain sight, and what did you still decide yourself?
- mwclient stops with an `HTTPError` on a 429, partway through a list. What would you lose, and what does the chapter say to do?
- Week 3 was about APIs that closed. Wikipedia's has stayed open. What keeps it open, and what would your project lose if it closed?

Step 17 — What did we learn? (3 minutes)

In a few sentences, as comments:

- Steps 5 and 16 counted 2023's edits two ways. Did they agree, and what would it mean if they didn't?
- In Step 13, how much of the article's traffic was people? What would a study that used `all-agents` have gotten wrong?
- Which of this week's numbers would differ if you ran the notebook next year, and which wouldn't? Why?
- What did the saved copies let you do, and what couldn't they tell you?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, choose an article and get its pageviews: Step 9's request, by hand, and Step 13's `get_pageviews()`, the chapter's function; end the window before today.
- Step 2, find the spike: Step 10's `idxmax()`, and its warning about UTC days.
- Step 3, get the revision history: Step 4's two pages by hand, and Step 5's `get_revisions()`.
- Step 4, profile the editors: Step 8's `nunique()` and `value_counts()`. Your table will have the `user` column, which the course's saved copy replaced with numbers.
- Step 5, align edits with attention: Step 11, which sums daily views by month and drops the edits' time zone before comparing.
- Step 6, map the neighborhood: Steps 14 and 15, where `page_list()` got the links and `query_all()` what links here; `page_list()` takes `"categories"` with `"cllimit"` and `"langlinks"` with `"lllimit"` the same way.
- Step 7, interpret: the Discuss questions after Steps 8, 10 and 11.

```
# Your answers here, as comments
# 1. They agreed: mwclient counted 91 edits in 2023, and so did Monday's table. If they hadn't,
 ↪  one had
#    stopped early or counted a different window, or an edit had been deleted since the table was
 ↪  saved.
# 2. Most of it, most months: in September 2026, 14,000 of 19,120 views (73%). But in August 2026
 ↪  people
#    were only 51%, when spiders read the article 9,362 times. A study using all-agents would
 ↪  have read
#    August's crawlers as readers' interest, and its counts from before 2025 mix in bots that
 ↪  Wikimedia
#    now calls automated.
# 3. The fixed windows don't change: 2,988 edits up to 2026-09-30; 2,099 days and 1,367,292
 ↪  views; the
#    peak, 16,767 views on 2023-09-17. What describes the article today does: 95,029 bytes on
 ↪  2026-11-10
#    (the chapter had 95,273), 652 links (650), 3,314 pages that link here (3,294), 72 redirects.
 ↪  Even
#    the pageviews can change, if Wikimedia reclassifies its traffic again.
# 4. They let Parts 2 and 3 start without a request, and let anyone who was refused carry on.
 ↪  They can't
```

```
#    show anything after 2026-11-10, and the course's copy has no names or edit summaries: we can
 ↪  count
#    the editors, but not say who they are or why they edited.
```

Friday: your portfolio checkpoint (15 minutes)

The last 15 minutes go to your project. This week's checkpoint is **[the instructor names it here, with its due date]**.

In fall 2026, week 10 had no checkpoint: its Sunday was the due date of the final project's **proposal**, which asks for a research question; your data source or sources and your method; and ethics and feasibility, with one record retrieved. If your project reads from an API, use the time to retrieve that record here, where you can ask, the way Step 1 did: print the status and the first 200 characters before reading anything.